Rapport de Projet Personnel : **Système de Surveillance Automatisée des Logements CROUS**

Hicham EL KHAZROUNI

Août 2025

1 Introduction

Dans un contexte de forte tension locative, la réactivité est le facteur déterminant pour obtenir un logement CROUS. La méthode manuelle, consistant à rafraîchir la page de recherche, s'avère chronophage et inefficace. L'objectif de ce projet était de concevoir un script capable de surveiller le portail en temps réel et d'envoyer des notifications instantanées sur mobile dès l'apparition d'une offre correspondant à des critères géographiques précis.

2 Démarche Technique et Évolution

2.1 Première approche : Le scraping statique (BeautifulSoup)

Initialement, j'ai développé une solution basée sur la bibliothèque `BeautifulSoup`. Cependant, j'ai rapidement identifié deux obstacles majeurs :

- **Contenu Dynamique :** Le site du CROUS utilise du JavaScript pour charger les annonces après le chargement initial de la page. Les requêtes HTTP classiques ne récupéraient donc qu'un squelette de page vide.
- **Détection Anti-Bot :** Sans gestion de session avancée, le serveur bloquait rapidement les requêtes répétitives.

2.2 Solution finale : Automatisation avec Selenium

Pour pallier ces limites, j'ai migré vers **Selenium**. Cet outil m'a permis de simuler un véritable navigateur, de laisser le JavaScript s'exécuter et d'interagir avec les éléments de la page.

- **Gestion des Sessions :** L'utilisation de cookies de connexion et de User-Agents réalistes a permis de simuler un comportement humain et de maintenir une session active.
- **Parsing hybride :** J'ai combiné Selenium pour la navigation et BeautifulSoup pour l'extraction rapide des données une fois le code source dynamique récupéré.

3 Fiabilité et Robustesse (Resilience by Design)

L'une des contraintes fortes était de laisser le script tourner en autonomie totale pendant une période de vacances. Un crash du programme aurait signifié l'arrêt de la surveillance.

3.1 Gestion des erreurs et auto-récupération

J'ai implémenté une structure robuste basée sur des blocs `try-except` imbriqués :

— En cas de crash du navigateur ou d'erreur réseau, le script ne s'arrête pas. Il ferme l'instance corrompue du driver, attend un délai de sécurité, puis relance automatiquement une nouvelle session.

— **Logging :** Un système de logs horodatés enregistre chaque événement (succès, erreur, redémarrage), permettant un diagnostic post-exécution.

3.2 Système de Notification

Pour l'alerte mobile, j'ai choisi l'**API ntfy**. Sa simplicité d'intégration via des requêtes POST et sa gratuité en ont fait l'outil idéal pour recevoir des alertes en temps réel sur mon téléphone sans avoir à gérer un serveur complexe de messagerie.

4 Configuration et Utilisation

Le script est conçu pour être adaptable. Pour le déployer sur une nouvelle machine :

1. **Paramétrage :** Modifier les chemins d'accès locaux (logs, drivers) indiqués en commentaires dans le code.
   (ligne 31, 33, 40, 162, 166, 200, 213, à modifier)
   (ligne 144 à mettre en commentaire si pas envie de notification toutes les heures)
2. **Authentification :** Importer les cookies de session du portail CROUS pour valider l'identité utilisateur.

   Pour récupérer la valeur du cookie de connexion il vous faut vous connecter au site dans un premier temps

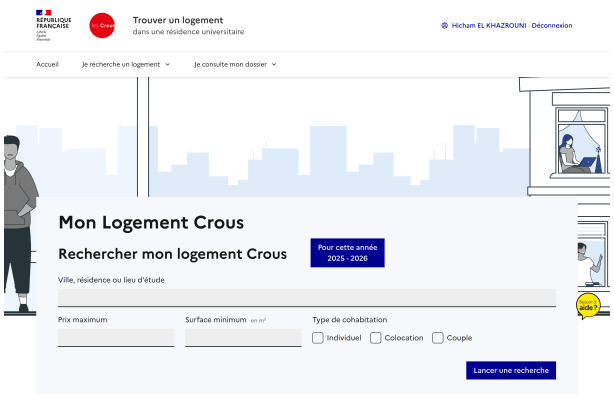

FIGURE 1 – En haut à droite on vérifie qu'on est connecté

Puis appuyez sur F12 pour ouvrir la page d'outils de développement (Developer Tools) et rechercher la cookies dans application/cookies/lapage puis copié la valeur du cookie et remplacez la dans le script

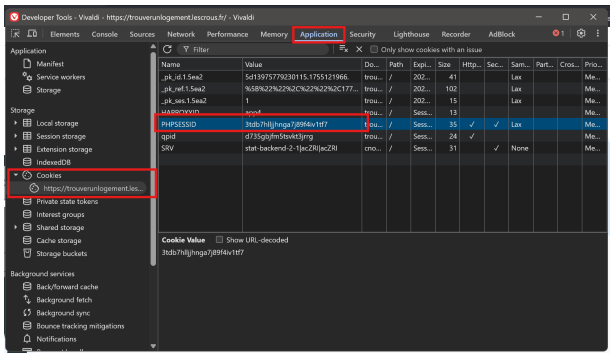

FIGURE 2 – Page d'outils de développement où est le cookie à copier

Cette étape est à refaire à chaque fois que vous relancez le script si la session n'est plus valide.

3. **Lancement :** Le script tourne en boucle infinie avec un intervalle de vérification configurable.

5 Compétences Acquises

Ce projet m'a permis de consolider plusieurs piliers du développement :

— **Autonomie :** Capacité à identifier l'échec d'une technologie et à apprendre un nouvel outil (Selenium) de manière autodidacte pour résoudreun problème concret.

— **Web Scraping :** Compréhension du DOM dynamique, des requêtes asynchrones et des mécanismes de protection des sites web.

— **Fiabilisation logicielle :** Conception de programmes "résilients" capables de fonctionner sans intervention humaine sur le long terme. Avec logs pour analyse post-exécution.

6 Validation et Résultats

Le script a été testé sur une période de 48 heures. Il a réussi à identifier plusieurs annonces dès leur mise en ligne.

— **Rapidité :** Notification reçue moins de 20 secondes après l'apparition de l'offre.

— **Fiabilité :** Aucun crash non récupéré constaté après l'implémentation de la gestion d'erreurs robuste.

7 Conclusion

Bien que je continue d'apprendre les subtilités du Web Scraping, ce projet est un succès fonctionnel. Il prouve qu'un outil automatisé bien conçu peut transformer une tâche fastidieuse en un flux de données automatisé et exploitable, répondant à un besoin concret de la vie étudiante.